

Exercise 13 — Code Readability Challenge

Exercise Description

Choose one or more of the sample-code examples, run the unit tests to understand the code, apply the appropriate readability prompt, refactor, and **re-run the tests to ensure the refactored code still works**. Reflect on readability improvements.

I completed both **Python** examples: - **Example 2 — Missing Documentation:** `calculator.py` (compound-interest `calculate`) + `test_calculator.py`. - **Example 4 — Poor Formatting and Structure:** `discount.py` + `test_discount.py`.

Example 2 — Missing Documentation (`calculate`)

Readability changes: added a full docstring (purpose, args with units, return dict), renamed `result` → `balance`, and named the year-boundary condition (`is_year_boundary`).

Behaviour was deliberately left identical — this is a documentation exercise, not a logic change.

Critical-eye finding (important): the starter **test contradicts the starter code**.

`test_with_additional_contributions` expects `final_amount == 2234.51`, but the provided `calculate` (and therefore my behaviour-preserving refactor) returns **2239.52**. I verified this by running the *original* untouched code:

```
original final_amount for (1000, 5, 3, 500): 2239.52    (test expects 2234.51)
```

So this test fails on the starter code before any refactoring. The ~5.01 gap comes from how the yearly `additional` contribution is timed inside the compounding loop. I chose **not** to silently change the compounding logic to force the number to match — that would be altering behaviour based on a guess. The right senior move is to flag the discrepancy for the exercise author and let the *intended* formula be confirmed. The other three calculator tests pass.

Example 4 — Poor Formatting and Structure (`discount`)

Readability changes: expanded the semicolon-packed one-liners into structured code; extracted `_cart_total`, `_meets_minimum`, `_promo_discount`, and `_status_discount`; documented that promotions and the loyalty discount **do not stack** (best single discount wins) and that a 'shipping' promo is a side effect setting `free_shipping`.

Result: all 6 discount tests pass — behaviour preserved.

Latent bug noted (not triggered by tests): the shipping branch uses `total >= promo['min_purchase']` without the `None` guard the other branches have, so a shipping

promo with `min_purchase=None` would raise `TypeError`. The tests always pass a numeric minimum, so it doesn't surface — but it's worth hardening.

Test results

```
calculator: Ran 4 tests – 3 pass, 1 pre-existing failure (starter test vs starter code mismatch)
discount   : Ran 6 tests – OK
```

Reflection Questions

1. **How much easier is the code to understand now?** Substantially — `discount` went from a dense semicolon block to four named helpers; `calculate` now states its units and return contract.
 2. **What did the AI catch that I missed?** The value of naming the year-boundary condition, and prompting a check of the `None`-guard asymmetry in the shipping branch.
 3. **What did I catch that the AI missed?** The starter test/code numeric mismatch in `calculate` — only visible by actually running the original, not by reading it.
 4. **Which improvement had the biggest impact?** Extracting `_promo_discount / _status_discount` — it turns "what does this branch do?" into self-describing function names.
 5. **What readability patterns will I reuse?** Name boolean conditions; extract per-case helpers from long if/elif chains; document non-obvious rules (non-stacking discounts) at the function level; and always *run* the tests before trusting them.
-

Submission for Exercise 13 — Code Readability Challenge

1. **Original code and refactored versions:** `calculator.py` (documented `calculate`) and `discount.py` (restructured with helpers); originals were the starter snippets.
2. **Tests still pass after changes:** `discount` — all 6 pass; `calculator` — 3/4 pass, with the 4th being a **pre-existing** starter mismatch (original code returns 2239.52 vs the test's 2234.51), proven by running the untouched original. My refactor preserved behaviour exactly.
3. **Readability issues the AI caught that I'd missed:** naming the year-boundary condition; the missing `None` guard in the shipping branch.
4. **Readability issues I caught that the AI missed:** the starter test-vs-code numeric inconsistency in `calculate` (found only by executing the original).

5. Reflection: biggest-impact change was extracting named per-case helpers from long if/elif chains; key lesson is to **run** the tests before trusting them — a "readability" task surfaced a real correctness discrepancy in the provided materials.